

Ansible

Ordoñez Figueroa Maria Fernanda

Facultad de Ingeniería, UNAM

6 de septiembre de 2025

- 1 ¿Tu Proyecto Necesita Ansible?
 - ¿En qué está escrito Ansible?
- 2 Primeros Pasos con Ansible
 - Guía rápida para aprender Ansible
- 3 Ejemplos Prácticos de los Beneficios de Ansible
- 4 Conclusión

- ¿Tengo que realizar tareas repetitivas?
- ¿Mi proyecto involucra a más de una persona?
- ¿Quiero que el despliegue sea rápido y sin errores?
- ¿Mi proyecto podría crecer en el futuro?

¿En qué está escrito Ansible?

- **Lenguaje base:** Ansible está desarrollado principalmente en Python.
- **Infraestructura:** usa librerías de Python y módulos que permiten conectarse a servidores (SSH, API, etc.).
- **Playbooks:** la forma de "programar" con Ansible no es con Python directo, sino con YAML (Yet Another Markup Language), que es un formato de texto legible y estructurado para describir tareas.

- 1 **Instala Ansible:** con `pip install ansible`.
- 2 **Crea un Inventario:** un archivo de texto con servidores.
- 3 **Escribe un Playbook:** empieza con algo simple.
- 4 **Ejecuta el Playbook:** con `ansible-playbook -i hosts archivo.yml`.

Guía rápida para aprender Ansible

1 Fundamentos previos

- Linux/Unix: comandos básicos, permisos, rutas.
- SSH: conexión entre máquinas.
- YAML: sintaxis (listas, diccionarios, indentación).

2 Primeros pasos en Ansible

- Instálalo en tu máquina (ej. `pip install ansible` o usando tu gestor de paquetes).
- Configura un archivo `hosts` (inventario) con los servidores donde quieres actuar.
- Prueba con ad-hoc commands: `ansible all -i hosts -m ping`

3 Playbooks

- Aprende a escribir playbooks en YAML.
- Usa módulos de Ansible (como `apt`, `yum`, `copy`, `service`, etc.).
- Haz un playbook sencillo (instalar un paquete, copiar un archivo).

4 Roles y buenas prácticas

- Organiza tareas en roles reutilizables.
- Usa variables (`vars`) y plantillas (templates con Jinja2).
- Maneja handlers (acciones que se ejecutan cuando algo cambia).

5 Escalar y profesionalizar

- Ansible Galaxy (repositorio de roles listos).

1. Automatización del Entorno de Desarrollo

- Cada desarrollador instala manualmente Python, Django y PostgreSQL.
- Esto genera problemas de compatibilidad entre versiones.
- Con Ansible, un solo archivo configura todo el entorno.

Ejemplo: dev_environment.yml

dev_environment.yml

```
---  
- hosts: localhost  
  connection: local  
  tasks:  
    - name: Ensure Python 3.9 is installed  
      become: yes  
      apt:  
        name: python3.9  
        state: present  
  
    - name: Install pip dependencies  
      pip:  
        requirements: /path/to/your/project/requirements.txt  
  
    - name: Ensure PostgreSQL client is installed  
      become: yes
```


2. Despliegue Rápido y Confiable

- El despliegue manual puede fallar si se olvida un paso.
- Con Ansible, un playbook ejecuta todo en el orden correcto.
- Permite repetir despliegues sin errores.

Ejemplo: deploy.yml

deploy.yml

```
---
- hosts: web_servers
  tasks:
    - name: Git pull latest code
      ansible.builtin.git:
        repo: 'https://github.com/your-repo.git'
        dest: '/var/www/my-app'
        version: main

    - name: Install dependencies
      ansible.builtin.pip:
        requirements: '/var/www/my-app/requirements.txt'

    - name: Run database migrations
      ansible.builtin.shell: 'python3 /var/www/my-app/manage.py
                             migrate'
```

3. Mantenimiento y Escalamiento Futuro

- Cuando la app crece, se añaden nuevos servidores.
- Con Ansible, basta con agregarlos al inventario.
- El mismo playbook asegura que todos estén configurados igual.

Ejemplo: hosts.ini

hosts.ini

```
[web_servers]
server1.yourdomain.com
server2.yourdomain.com
```

- Ansible asegura entornos consistentes.
- Facilita despliegues rápidos y confiables.
- Escala fácilmente a múltiples servidores.
- Te permite enfocarte en lo más importante: **desarrollar software de calidad.**